Contents

Security & DevSecOps — Deep Dive Notes **1**
1. DevSecOps Philosophy . . . 1
2. Security Toolchain Per Pipeline Stage . . . 1
   - Pre-commit (Developer Machine) . . . 1
   - Source / CI Stage — SAST (Static Analysis) . . . 1
   - SCA — Dependency Scanning . . . 2
   - Container Security Scanning . . . 2
   - IaC Security Scanning . . . 2
   - DAST — Dynamic Testing (Running App) . . . 2
3. Secrets Management . . . 2
   - Why Not Environment Variables . . . 2
   - HashiCorp Vault — Architecture . . . 3
   - AWS Secrets Manager Best Practices . . . 3
4. Kubernetes Security Hardening . . . 3
   - Security Context (Per Pod/Container) . . . 3
   - Pod Security Admission (Replaces deprecated PodSecurityPolicy) . . . 4
   - RBAC Hardening . . . 4
   - Network Policies — Micro-Segmentation Model . . . 4
   - Admission Controllers Worth Knowing . . . 4
5. Supply Chain Security (SLSA Framework) . . . 5
   - SLSA Levels . . . 5
   - Cosign — Image Signing and Verification . . . 5
   - SBOM (Software Bill of Materials) . . . 5
6. Runtime Security . . . 5
   - Falco — Cloud-Native Runtime Security . . . 5
7. Compliance Frameworks at a Glance . . . 6
   - Automation for Compliance . . . 6
- Resources . . . 6

Security & DevSecOps — Deep Dive Notes

1. DevSecOps Philosophy

Shift-Left Security — integrate security at every pipeline stage, not at the end.

```
TRADITIONAL:  [Dev] → [Dev] → [QA] → [Security] → [Deploy]   ← Security is a bottleneck
DEVSECOPS:    [Dev+Sec] → [Build+Scan] → [Test+Scan] → [Deploy+Monitor]
```

Cost multiplier of fixing bugs: - Design phase: 1x - Development: 10x - Testing: 100x - Production: 10,000x+

---

2. Security Toolchain Per Pipeline Stage

Pre-commit (Developer Machine)

- **gitleaks** — scan for hardcoded secrets (API keys, passwords, tokens)
- **git-secrets** — prevent AWS credentials from being committed
- **detect-secrets** — Yelp's secret scanning tool
- **pre-commit framework** — runs hooks before every `git commit`

Source / CI Stage — SAST (Static Analysis)

Test code without running it. Finds logic bugs and known vulnerability patterns.

Tool	Best For	Key Checks
SonarQube	All languages	OWASP Top 10, code coverage gates
Semgrep	Multi-language custom rules	SQL injection, XSS, dangerous functions
Bandit	Python	os.system, hardcoded passwords, pickle
CodeQL	GitHub Actions native	Deep data-flow analysis
SpotBugs + FindSecBugs	Java	Deserialization, injection

SCA — Dependency Scanning

Find CVEs in third-party libraries before they reach production.

Tool	Strength
Snyk	Largest DB, auto-fix PRs, license checks
Dependabot	GitHub native, free, auto-PRs
Trivy	Covers containers + filesystems + IaC
OWASP Dependency-Check	Open source, Java/JS/Python

Container Security Scanning

```
trivy image --severity HIGH,CRITICAL myapp:1.0.0
grype myapp:1.0.0 --fail-on high

Rules:
  - Never use :latest tag in production
  - Pin digest: FROM python@sha256:abc123...
  - Use distroless or alpine base images
  - Rebuild base images weekly for OS patches
  - Fail CI on any CRITICAL CVE
```

IaC Security Scanning

```
checkov -d ./terraform       -- misconfigs in Terraform
tfsec ./terraform            -- Terraform-specific rules
kube-score score ./k8s/      -- K8s manifest quality
kubesec scan ./k8s/pod.yaml  -- K8s security scoring
```

DAST — Dynamic Testing (Running App)

Tests the live application for vulnerabilities.

- **OWASP ZAP**: Free, CI-friendly, passive + active scanning
- **Burp Suite**: Industry standard for manual pen testing
- **Nuclei**: Template-based fast scanner (thousands of templates)
- Run against staging environment before promoting to production

---

3. Secrets Management

Why Not Environment Variables

- Visible in `/proc/PID/environ` — any process can read
- Often logged accidentally by frameworks
- Passed to all child processes
- Require restart to rotate

HashiCorp Vault — Architecture

```
Auth Methods (how clients authenticate):
  - Kubernetes  →  ServiceAccount token → Vault token
  - AWS IAM     →  Instance/role identity → Vault token
  - OIDC        →  GitHub, Okta, Google → Vault token
  - AppRole     →  Role ID + Secret ID (CI systems)

Secret Engines (how secrets are stored/generated):
  - KV v2         →  Static key-value with versioning
  - Database      →  Dynamic credentials (Postgres, MySQL)
  - PKI           →  Certificate authority, issue TLS certs
  - AWS           →  Dynamic IAM credentials
  - SSH           →  OTP or CA-signed SSH certificates

Kubernetes Integration Patterns:
  A) Vault Agent Injector (Sidecar):
     Pod annotation → Vault injects secret as file in /vault/secrets/
     App reads file — never touches K8s secrets

  B) External Secrets Operator:
     ExternalSecret CRD → syncs from Vault/AWS SM → K8s Secret
     Easier but secrets still live in etcd (encrypt etcd at rest)

  C) CSI Secrets Store Driver:
     SecretProviderClass → mounts secrets directly as volumes
     No K8s Secret object created
```

AWS Secrets Manager Best Practices

```
1. Enable automatic rotation (Lambda rotators for RDS, Redshift)
2. Use resource policies for cross-account access
3. Enable CloudTrail logging for all GetSecretValue calls
4. Set ResourcePolicy to deny access from non-approved roles
5. Use SecretString for small secrets, reference S3 for large payloads
6. Tag secrets: Environment, Team, Service (for cost allocation + access control)
7. Use conditions in IAM: aws:ResourceTag/Environment = production
```

---

4. Kubernetes Security Hardening

Security Context (Per Pod/Container)

```yaml
# Pod-level security context
securityContext:
  runAsNonRoot: true
  runAsUser: 1001
  runAsGroup: 1001
  fsGroup: 1001
  seccompProfile:
    type: RuntimeDefault   # uses container runtime seccomp profile
  sysctls: []              # no unsafe sysctls

# Container-level security context
containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
```

```
      capabilities:
        drop: [ALL]        # drop ALL capabilities
        add: []            # add only what's strictly needed
```

Pod Security Admission (Replaces deprecated PodSecurityPolicy)

```
Enforce at namespace level via labels:

  privileged  → no restrictions (only for system namespaces)
  baseline    → prevents known privilege escalations
  restricted  → maximum hardening (non-root, no capabilities, read-only FS)

Apply to namespace:
  pod-security.kubernetes.io/enforce: restricted
  pod-security.kubernetes.io/warn: restricted
  pod-security.kubernetes.io/audit: restricted
```

RBAC Hardening

```
Key rules:
  1. No wildcard verbs ("*") or resources ("*") in production
  2. No cluster-admin for service accounts
  3. Namespace-scoped Roles > ClusterRoles for most cases
  4. Audit quarterly: who has access to what
  5. Use kubectl auth can-i --list --as=system:serviceaccount:ns:sa
  6. Disable default ServiceAccount token automounting
     automountServiceAccountToken: false  (in ServiceAccount spec)
```

Network Policies — Micro-Segmentation Model

```
Step 1: Default deny all (namespace-wide)
  Both ingress and egress default deny

Step 2: Whitelist per service
  Allow ingress from ingress-controller to web tier on 443
  Allow ingress from web to api on 8080
  Allow ingress from api to database on 5432
  Allow all pods to DNS (kube-dns) on port 53
  Allow monitoring to scrape metrics on 9090

Step 3: Egress restrictions
  Deny internet egress for database pods
  Allow controlled outbound for specific external APIs
```

Admission Controllers Worth Knowing

```
Mutating webhooks (modify objects):
  - Istio Sidecar Injector — adds Envoy proxy
  - Vault Agent Injector — adds Vault agent sidecar
  - Pod Security Policy (deprecated) / Kyverno mutate

Validating webhooks (accept/reject):
  - OPA/Gatekeeper — policy-as-code
  - Kyverno — policy engine (simpler than OPA for K8s)
  - ImagePolicyWebhook — enforce trusted registries only

Built-in:
  - LimitRanger — enforce resource limits
  - ResourceQuota — enforce namespace quotas
```

```
  - NamespaceLifecycle — prevent ops on deleting namespaces
```

---

5. Supply Chain Security (SLSA Framework)

SLSA Levels

```
SLSA 0 → No requirements
SLSA 1 → Build scripted, provenance available
SLSA 2 → Version control + hosted build service + signed provenance
SLSA 3 → Hardened build, provenance authenticated, non-forgeable
SLSA 4 → Hermetic + reproducible builds, two-party review

Goal: Protect against dependency confusion, build tampering, source compromises
```

Cosign — Image Signing and Verification

```
# Sign an image (keyless, OIDC-based — works in GitHub Actions)
cosign sign --yes registry.io/org/myapp:v1.2.3

# Verify at deployment time
cosign verify \
  --certificate-identity=github-actions@company.iam.gserviceaccount.com \
  --certificate-oidc-issuer=https://token.actions.githubusercontent.com \
  registry.io/org/myapp:v1.2.3

# Generate SBOM and attach to image
syft registry.io/org/myapp:v1.2.3 -o spdx-json > sbom.json
cosign attach sbom --sbom sbom.json registry.io/org/myapp:v1.2.3
```

SBOM (Software Bill of Materials)

- Complete inventory of all components in a software artifact
- Formats: SPDX, CycloneDX
- Required by US Executive Order 14028 for federal software
- Tools: Syft (generation), Grype (vulnerability scanning from SBOM)

---

6. Runtime Security

Falco — Cloud-Native Runtime Security

```
What it detects (via eBPF syscall monitoring):
  - Shell spawned inside container
  - Sensitive file reads (/etc/shadow, /etc/kubernetes/admin.conf)
  - Outbound connections to unexpected destinations
  - Privilege escalation attempts
  - Crypto mining behavior patterns
  - Container drift (unexpected binary execution)

Deployment: DaemonSet on every node
Output: Syslog, webhook, Slack, Falcosidekick for routing

Example rule concept:
  Alert when: process != expected binary list runs inside container
  Alert when: write to /etc/ inside running container
  Alert when: container makes unexpected network connection
```

---

7. Compliance Frameworks at a Glance

Standard	Industry	Key DevOps Requirements
SOC 2 Type II	SaaS/Cloud	Audit trails, access control, change management, annual audit
PCI-DSS v4	Payments	Network segmentation, encryption, no default passwords, pen testing
HIPAA	Healthcare (US)	PHI access controls, audit logs, encryption, BAA agreements
ISO 27001	All industries	ISMS, risk management, security policies, annual audit
GDPR	EU/handling EU data	Data mapping, consent, right to erasure, 72hr breach notification
FedRAMP	US Federal Cloud	Continuous monitoring, NIST 800-53 controls, ATO authorization

Automation for Compliance

```
Policy as Code:  OPA/Gatekeeper, Kyverno, Sentinel (Terraform)
Audit Logging:   CloudTrail + SIEM (Splunk/OpenSearch), K8s audit logs
Vulnerability:   Trivy + weekly scans + patching SLA dashboards
Access Reviews:  Automated quarterly reports from IAM + K8s RBAC
Evidence:        Automated compliance evidence collection (Drata, Vanta)
```

Resources

- https://owasp.org/www-project-top-ten/ — OWASP Top 10
- https://cheatsheetseries.owasp.org/ — OWASP Cheat Sheets
- https://slsa.dev/ — Supply chain security levels
- https://www.cisecurity.org/cis-benchmarks — CIS Benchmarks (free)
- https://developer.hashicorp.com/vault/docs — HashiCorp Vault
- https://falco.org/docs/ — Falco runtime security
- https://kyverno.io/docs/ — Kyverno policies
- https://aquasecurity.github.io/trivy/ — Trivy scanner